University of Inland Norway

Faculty of Audiovisual Media and Creative Technologies

Bachelor thesis

Saving the Game: Consistent Offline-Cloud State Synchronization in Survival Video Games

Lagre spillet:
Konsistent synkronisering av lokal- og skybasert tilstand i overlevelsesspill

B2VISIM Bachelor in Game Technology and Simulation

SPIS2900 Bachelor thesis

Hans Alan Whitburn Haugen

April 19, 2026

Abstract

Many survival games store their world entirely on remote servers, which means the moment a player loses their internet connection, progress stops. This thesis looks at how that limitation can be overcome — specifically, how a game can keep running offline and then safely merge what happened back into the shared world when the player reconnects.

The core idea is to record each player action as a small, self-contained command object rather than immediately writing to the server. While offline, these commands queue up locally. When connectivity returns, they are replayed against the authoritative server in order. The tricky part is deciding what to do when the server's world has moved on in the meantime: another player may have demolished the same building, or claimed the same territory. The thesis addresses this through a conflict resolution strategy that classifies commands by type — some can always be applied regardless of what changed, some need a version check, and some must simply be declined and reported back to the player.

A prototype was built in C++ to validate the design, including a SQLite-backed offline database and a durable command log that survives crashes. All components are tested with 31 automated tests.

Sammendrag

Mange overlevelsesspill lagrer hele spillverdenen på eksterne servere, noe som betyr at fremgangen stopper i det øyeblikket spilleren mister internettforbindelsen. Denne oppgaven undersøker hvordan den begrensningen kan overkommes — nærmere bestemt hvordan et spill kan fortsette å kjøre uten nett og deretter trygt slå sammen det som skjedde med den delte verdenen når spilleren kobler seg til igjen.

Kjerneide en er å registrere hver spillerhandling som et lite, selvstendig kommandoobjekt i stedet for å skrive direkte til serveren. Mens spilleren er frakoblet, samles disse kommandoene opp lokalt. Når forbindelsen er gjenopprettet, spilles de av mot den autoritative serveren i riktig rekkefølge. Det vanskelige er å avgjøre hva som skal skje når serverens verden har endret seg i mellomtiden: en annen spiller kan ha revet ned den samme bygningen eller inntatt det samme territoriet. Oppgaven løser dette gjennom en konflikthåndteringsstrategi som klassifiserer kommandoer etter type — noen kan alltid anvendes uavhengig av hva som har endret seg, andre krever en versjonssjekk, og noen må rett og slett avvises og rapporteres tilbake til spilleren.

En prototype ble bygget i C++ for å validere designet, inkludert en SQLite-basert frakoblet database og en holdbar kommandologg som overlever krasj. Alle komponenter er testet med 31 automatiserte tester.

Contents

1 Introduction **1**

2 Typical Challenges in Persistence Systems **2**
2.1 Limitations of online server-based systems 3
2.2 Dependence on Continuous Connectivity 3
2.3 Absence of Local Persistence Authority 3
2.4 Lack of Divergence Handling Mechanisms 3
2.5 Scalability vs. Flexibility Trade-off 4

3 Proposed Hybrid Persistence Architecture **5**
3.1 Command-Based State Replication 5
3.2 Implementation considerations 6
3.3 Conflict Resolution Strategy 6
3.3.1 Conflict Detection via Entity Versioning 6
3.3.2 Command Classification 7
3.3.3 Resolution Policies 7
3.3.4 Rejection Manifest 8
3.4 Integration Possibilities 8
3.5 Local Persistence via SQLite 8
3.6 ORM (Object-Relational Mapping) 8
3.7 Prototype Implementation 9

4 Discussion **11**

5 Conclusion **13**

Listings

1 DBI interface (abstract base class) 2
2 Command structure and local command log 5
3 Server-side command validation and conflict resolution 7

1 Introduction

This thesis considers ways to develop the persistence system of a modern large-scale survival multiplayer game. The designs are fitting for a persistent multiplayer world shared by thousands of players. Solutions are designed to support large-scale online survival games which need to store player progression, world state, and faction dynamics. The solutions are not intended to store or transmit ephemeral information such as player position or combat-related replication, instead only long-term storage data of the kind which can typically be stored in central cloud-hosted SQL databases are of consideration.

The suggested designs are intended to provide ways to support both offline and online functionality. Having the ability to make a game run offline is important, as on console platforms access to online multiplayer features typically requires a paid subscription service. A significant portion of console players therefore do not participate in online multiplayer environments (Baker, 2025). Playing with friends should be supported via local cooperative modes while preserving core gameplay mechanics.

This requirement introduces architectural challenges which need to be addressed sensibly. Games on the market which support persistent game state stored exclusively in cloud-based SQL databases assume continuous server connectivity. Supporting offline gameplay necessitates a hybrid architecture in which persistent world state can be stored locally, potentially using embedded database solutions such as SQLite which can later be synchronized with the authoritative cloud-based server infrastructure.

Many gamers prefer to play offline, but might want to connect online as they advance in the persistent world. Collaboration with other players is often necessary in survival games to navigate the harsh environment. Political dynamics and faction-based conflict play a central role in survival games, as groups of players compete for control and influence within the game's persistent world. Designs that allow users to transition online after initial offline sessions are explored.

This thesis investigates the following research statement: How can synchronization algorithms for persistent game state be designed and evaluated with respect to consistency, conflict resolution, and scalability in large-scale survival multiplayer games?

2 Typical Challenges in Persistence Systems

Game worlds consist of actors that may own inventory items, buildings, vehicles, navigation markers, abilities, and quest progression. Persistent game state is commonly stored in centralized persistence systems backed by SQL databases.

Such systems follow a server-based architecture in which all state-modifying actions are processed through a centralized online infrastructure. Player actions are buffered temporarily before being committed to online servers for persistence. Multiple server instances may run concurrently; however, at any given time, only one server instance should have authoritative control over a specific player. Player actions can be executed as database transactions using SQL on the authoritative server.

One possible approach is to apply the Service Locator pattern to provide separate implementations for the communication service which is responsible for generating and receiving SQL commands. This service is hereafter referred to as the Database Interface (DBI). Rather than generating raw SQL queries directly from the game client, the DBI invokes predefined SQL functions. Each function executes as a transactional unit and can be rolled back if any command within the transaction fails. Consequently, every state-modifying action is encapsulated as an atomic database operation.

The DBI is designed to support atomicity, consistency, and idempotency. If a failure occurs, or if the system is uncertain whether a transaction completed successfully, the DBI may safely retry the corresponding function without risking duplicate or inconsistent state changes. Additional challenges arise under network partitions or concurrent updates, where conflicting replicas may emerge despite transactional guarantees.

By applying the Service Locator pattern, the DBI API can be swapped out easily for online and offline sessions. On online sessions, the DBI connects to an online backend SQL server. On offline sessions, there are many possible implementations. It could connect to a local SQLite database, or have a unique implementation which is explored more later.

A SQL-based DBI architecture assumes continuous server connectivity and treats the cloud-hosted SQL database as the single authoritative source of truth for all persistent game state.

While this architecture provides strong consistency guarantees in fully online environments, it inherently depends on centralized authority and does not account for state divergence in disconnected or offline scenarios.

Listing 1: DBI interface (abstract base class)

```
class IDatabaseInterface
{
public:
    virtual ~IDatabaseInterface() = default;
```

```
    virtual bool CreateBuilding(int32_t playerId, int32_t buildingTypeId,
                                float posX, float posY, float posZ) = 0;
    virtual bool DestroyBuilding(int32_t buildingId) = 0;

    virtual bool TransferItem(int32_t fromPlayerId, int32_t toPlayerId,
                              int32_t itemId, int32_t quantity) = 0;

    virtual bool CompleteQuest(int32_t playerId, int32_t questId) = 0;

    virtual bool UpdateFactionRelation(int32_t factionA, int32_t factionB,
                                       int32_t relationDelta) = 0;
};
```

2.1 Limitations of online server-based systems

Persistence architecture designed in this fashion operates under the assumption of continuous connectivity to a centralized cloud infrastructure. All persistent game state is stored in SQL databases hosted on backend servers, and the system operates under a server-authoritative model in which the cloud acts as the single source of truth.

While this model ensures strong consistency and transactional integrity in online multiplayer environments, it introduces several architectural limitations when extended to offline or hybrid scenarios.

2.2 Dependence on Continuous Connectivity

The system requires uninterrupted communication between the game client and backend services. Because all state-modifying actions are executed as remote transactions, gameplay cannot progress without server access. This dependency prevents offline play and limits deployment flexibility on platforms where online access may be restricted or subscription-based.

2.3 Absence of Local Persistence Authority

In the current online server-based model, the client does not maintain an authoritative local representation of persistent world state. Although temporary state may exist in memory during runtime, durable storage is exclusively managed by the cloud infrastructure. This design prevents local world progression in disconnected scenarios and complicates support for local cooperative modes.

2.4 Lack of Divergence Handling Mechanisms

An online-only architecture assumes a single linear progression of state changes. It does not account for concurrent or divergent modifications occurring in sepa-

rate environments (e.g., offline sessions modifying the same player or world data as the cloud). Consequently, the system lacks:

1. Version tracking mechanisms
2. Conflict detection strategies
3. Deterministic merge policies

Without these components, synchronization between locally modified state and cloud-based state cannot be performed safely.

2.5 Scalability vs. Flexibility Trade-off

The centralized server-authoritative design scales well for persistent multiplayer environments but sacrifices flexibility. Any extension toward hybrid offline–online play requires introducing distributed state management, which the current architecture was not designed to support.

3 Proposed Hybrid Persistence Architecture

3.1 Command-Based State Replication

To enable offline progression while preserving compatibility with the existing server-authoritative model, this thesis proposes a command-based persistence mechanism inspired by the Command Pattern as described in Design Patterns: Elements of Reusable Object-Oriented Software and further discussed in Game Programming Patterns (Nystrom, 2014).

Rather than directly modifying persistent state in the cloud, player actions are encapsulated as serializable command objects. Each command represents an intention to modify game state (e.g., create building, transfer item, complete quest) along with the necessary parameters to execute the modification.

During offline gameplay, commands are appended to a durable local command log instead of being transmitted immediately to the server. When connectivity is restored, the queued commands are transmitted sequentially to the authoritative backend infrastructure and executed as transactional operations.

This approach introduces several advantages:

1. Commands are naturally serializable and can be stored locally.
2. Replay of commands enables deterministic reconstruction of state.
3. The command log provides a history of offline actions.
4. Existing server-side transactional guarantees can remain unchanged.

By storing intent rather than derived state, the system avoids directly merging divergent world snapshots and instead synchronizes through ordered execution of state-modifying operations.

Listing 2: Command structure and local command log

```
struct Command
{
    std::string    type;              // e.g. "CreateBuilding", "TransferItem"
    int64_t        timestamp;         // UTC milliseconds at time of action
    std::string    idempotencyKey;    // UUID to detect duplicate replays
    std::string    params;            // JSON-serialized action parameters
};

class CommandLog
{
public:
    void Append(const Command& cmd);
    std::vector<Command> GetPending() const;
    void MarkFlushed(const std::string& idempotencyKey);
```

```
private:
    std::vector<Command> m_pending;
};
```

3.2 Implementation considerations

This system must be designed with integrity guarantees in mind. Since commands are generated client-side and stored locally, a malicious client could potentially forge or alter commands before transmission. Every action must therefore be validated on the server before it is committed, and any command that produces an invalid or impossible state transition must be rejected. Conflict detection, entity versioning, and per-type resolution policies are addressed in the following subsection.

Beyond conflict handling, two additional concerns must be managed. First, ordering guarantees: the command log must preserve the sequence in which actions were taken offline, and the server must replay them in that order to ensure deterministic results. Second, idempotency and duplicate detection: if a replay attempt fails partway through or a command is transmitted more than once, the server must be able to identify and skip already-applied commands. The idempotency key stored on each command serves this purpose.

A remaining open concern is command dependency on server-side state that changed during the offline session. If an offline command assumed the existence of an entity that was deleted by another player while the client was disconnected, the command cannot be applied even if its version check would otherwise pass. Handling such cases requires pre-condition checks on the server that go beyond version comparison, and the rejection manifest provides the mechanism for communicating these failures back to the client.

3.3 Conflict Resolution Strategy

When offline commands are replayed against the authoritative server, conflicts arise when the server state has diverged from the state against which those commands were originally generated. A command that was valid at the time of creation may produce an invalid or inconsistent state transition when applied to an evolved world.

3.3.1 Conflict Detection via Entity Versioning

Each persistent entity (e.g. a building, an inventory slot, or a faction) is assigned a monotonically increasing version number that is incremented with every committed state change. When a command is generated offline, the version of each affected entity is recorded alongside the command parameters. Upon replay, the server compares the recorded version against the current entity version. A mismatch indicates that the entity was modified after the command was issued, signalling a potential conflict.

This mechanism mirrors optimistic concurrency control (OCC) in traditional database systems, where transactions proceed without locking but are validated at commit time (Bernstein & Goodman, 1981).

3.3.2 Command Classification

Not all version mismatches require rejection. Commands can be classified by their conflict behaviour:

1. **Idempotent commands** — Quest completion, achievement unlocks, and similar one-time flags. These produce the same result regardless of how many times they are applied and can be safely replayed unconditionally.
2. **Commutative commands** — Additive operations such as resource collection or faction reputation deltas. Because the order of application does not affect the final value, these can be merged without conflict by summing the deltas.
3. **Non-commutative commands** — Destructive or positional operations such as demolishing a building or placing a structure on a specific grid cell. These depend on preconditions that may have been invalidated by independent server-side changes.

3.3.3 Resolution Policies

For each conflict class, a deterministic resolution policy is applied server-side before any command is committed.

Listing 3: Server-side command validation and conflict resolution

```
ReplayResult ReplayCommand(const Command& cmd, Database& db)
{
    Entity entity = db.GetEntity(cmd.entityId);

    // Idempotent: already applied, skip silently
    if (db.IsAlreadyApplied(cmd.idempotencyKey))
        return ReplayResult::Duplicate;

    // Commutative: apply delta regardless of version
    if (cmd.type == "ResourceDelta" || cmd.type == "ReputationDelta")
        return db.ApplyDelta(cmd);

    // Non-commutative: validate preconditions
    if (entity.version != cmd.expectedVersion)
        return ReplayResult::Conflict;

    if (!entity.MeetsPreConditions(cmd))
        return ReplayResult::Rejected;
```

```
    return db.Commit(cmd);
}
```

3.3.4 Rejection Manifest

Commands that fail validation are not silently discarded. Instead, they are collected into a rejection manifest that is returned to the client after the replay pass completes. The manifest records the command type, the reason for rejection, and the current server-side state of the affected entity. This allows the game to inform the player of which offline actions could not be applied and why, enabling the client to present meaningful feedback rather than silently losing progress.

3.4 Integration Possibilities

Currently, every action that should persist goes through the DBI. By extending the DBI into a factory that produces command objects, the system can be made to route actions in two ways depending on connectivity state: online commands are queued and transmitted to the server immediately as before, while offline commands are serialized and stored on disk, deferred until the player reconnects.

3.5 Local Persistence via SQLite

A direct approach to enabling offline functionality is to extend the existing Database Interface (DBI) abstraction with a complete SQLite-backed implementation. Rather than communicating with remote SQL servers, the DBI would execute equivalent operations against a locally embedded SQLite database during offline sessions.

SQLite is an embedded, serverless, ACID-compliant relational database engine that operates as a lightweight C/C++ library. Unlike other SQL implementations, it does not require a separate server process; instead, it runs within the host application and stores data in a single file on disk. This makes it well-suited for offline persistence in desktop and console environments.

By preserving the DBI abstraction layer, the system can maintain a consistent programming interface for persistence operations regardless of connectivity state. When offline, DBI calls would be routed to the SQLite backend. When online, calls would be directed to the existing SQL infrastructure.

This design minimizes disruption to the gameplay code, as the persistence boundary remains unchanged. However, it introduces challenges related to schema compatibility, transaction semantics, and synchronization between the local SQLite instance and the authoritative cloud database.

3.6 ORM (Object-Relational Mapping)

An alternative architectural approach considered was the introduction of an Object–Relational Mapping (ORM) system. An ORM provides a layer that

maps in-memory C++ objects directly to relational database structures, allowing persistent entities to be stored and retrieved without writing explicit SQL logic. In such a system, game objects (e.g., inventory items, buildings, or quests) would be automatically translated into database rows and reconstructed from relational data.

ORM frameworks are common in higher-level languages such as Python and Java, where reflection and dynamic typing simplify runtime type inspection and schema generation. In contrast, implementing a fully featured ORM in C++ presents additional complexity due to the lack of built-in reflection and the need for explicit type registration and serialization logic.

While an ORM could reduce manual database interaction and potentially simplify persistence logic, several factors limited its suitability for the current project. First, the existing architecture already relies on a stable Database Interface (DBI) abstraction connected to predefined SQL functions. Introducing an ORM would require significant restructuring of this persistence boundary. Second, the project required explicit support for SQLite in offline mode, meaning that any ORM solution would need to maintain compatibility across multiple database backends. Finally, time constraints and integration risks made a full ORM implementation impractical within the scope of this bachelor project.

For these reasons, the project retained the existing DBI abstraction and instead focused on extending it to support hybrid online–offline persistence rather than replacing it with a generalized object–relational mapping layer.

3.7 Prototype Implementation

A self-contained C++ prototype of the proposed architecture was developed alongside this thesis and is available at:

https://github.com/alanhaugen/SPIS2900-Bachelor

The prototype is located in the `app/` directory and is built with CMake. It depends only on Catch2, which is fetched automatically at configure time, and a system SQLite3 library. The implementation is organized as two libraries:

`offline_sync_lib` contains the core persistence abstractions:

- `IDatabaseInterface` — the abstract DBI used by game code, with virtual methods for all persistent game actions.
- `Command` and `CommandLog` — the in-memory command queue described in Section 3.1.
- `Database` — a lightweight in-memory entity store used for testing, with version tracking and idempotency key recording.
- `ReplayCommand` / `ReplayAll` — the conflict-resolution algorithm from Section 3.3, including the `RejectionManifest` type.

`offline_sync_sqlite_lib` contains the SQLite-backed implementations:

- `SQLiteDBI` — a concrete implementation of `IDatabaseInterface` that stores game state in a local SQLite file. It maintains game-domain tables for buildings, item inventories, completed quests, and faction relations. Item transfers execute inside an explicit SQLite transaction and are rejected atomically if the sender has insufficient inventory. Quest completion uses `INSERT OR IGNORE` to preserve idempotency. Faction relation updates use an UPSERT to accumulate signed deltas.
- `SQLiteCommandLog` — a durable replacement for the in-memory `CommandLog` that writes each command to a SQLite file before returning. The `idempotency_key` column carries a `UNIQUE` constraint so that a double-append is silently discarded at the database level. Commands survive process restarts and can be recovered on next open.

The prototype includes 31 Catch2 unit tests covering the command log, idempotency, commutative merging, non-commutative conflict detection, SQLite persistence, and a full offline-session integration scenario.

4 Discussion

Persistent state management in online multiplayer games can be implemented using a wide range of architectural strategies. These range from simple centralized database models to more complex distributed synchronization systems. Each approach introduces trade-offs in terms of implementation complexity, scalability, maintainability, and consistency guarantees.

A purely server-authoritative architecture, such as the one described in the existing system, provides strong consistency and simplifies conflict resolution by maintaining a single source of truth. However, this model assumes continuous connectivity and does not naturally support offline progression or hybrid play modes.

At the other end of the spectrum, snapshot-based synchronization models allow entire world states to be stored locally and merged with cloud-based state upon reconnection. While conceptually straightforward, snapshot merging introduces significant challenges in conflict detection, state reconciliation, and deterministic resolution of concurrent modifications. As the complexity of the world model increases—particularly in survival games with persistent player-owned structures and factional systems—state merging becomes increasingly difficult to reason about.

The command-based approach proposed in this thesis represents a middle ground between these extremes. By recording player intent as serialized commands rather than storing divergent state snapshots, synchronization can occur through ordered replay of operations. This reduces the need for complex structural merging and leverages existing transactional guarantees in the backend infrastructure.

Compared to a full object–relational mapping redesign or a distributed consensus-based architecture, the command pattern solution introduces relatively limited structural change to the existing system. It preserves the Database Interface abstraction and integrates naturally with the transactional SQL backend. At the same time, it enables offline progression through local command logging and deferred synchronization.

However, the approach is not without limitations. Command-based replication assumes that operations are deterministic and replay-safe. The conflict resolution strategy proposed here — entity versioning combined with per-type resolution policies — addresses the common cases, but edge cases remain. Commands that depend on entities deleted by other players during the offline session cannot be recovered by version comparison alone and must be caught by pre-condition checks during server-side validation. In sufficiently large-scale environments, the volume of pending commands and the breadth of pre-condition logic could also introduce replay latency that affects the reconnection experience.

Overall, the proposed architecture aims to balance simplicity, extensibility, and correctness. Rather than introducing a fundamentally new persistence

paradigm, it extends the existing transactional model in a way that supports hybrid connectivity while minimizing architectural disruption.

Having offline functionality support will prevent the game from having to be connected to the publisher in order to function. This is important for games as cultural products, making them available after companies shut down online services for their games (Ondruška et al., 2026).

5 Conclusion

This thesis investigated the following research question: How can synchronization algorithms for persistent game state be designed and evaluated with respect to consistency, conflict resolution, and scalability in large-scale survival multiplayer games?

The starting point was a server-authoritative persistence architecture in which all state changes are executed as transactional SQL operations against a centralized cloud backend. While robust in fully online environments, this model assumes continuous connectivity and does not support offline progression or hybrid play modes. Enabling offline functionality therefore requires architectural changes that preserve data integrity while allowing temporary divergence of state.

Several architectural strategies were considered. Direct local database mirroring through SQLite provides a familiar persistence interface during offline sessions but does not itself address how divergent state is reconciled upon reconnection. An object–relational mapping layer was also evaluated but rejected due to integration complexity and the risk of disrupting the existing persistence boundary.

The proposed solution is a command-based replication model. Rather than synchronizing complete world snapshots, the system records player intent as serializable commands that can be replayed deterministically against the authoritative server. This approach addresses the three dimensions of the research question as follows.

Consistency is maintained by treating the cloud server as the single authoritative source of truth. Offline commands are not committed until they have been validated and replayed server-side. The existing transactional guarantees of the SQL backend are preserved throughout.

Conflict resolution is handled through a combination of entity versioning, command classification, and per-type resolution policies. Idempotent commands are replayed unconditionally; commutative operations such as resource deltas are merged by summation; and non-commutative operations are validated against current server state before being accepted or rejected. Commands that cannot be applied are returned to the client in a rejection manifest rather than being silently discarded.

Scalability is supported by the architecture's reliance on the existing server-authoritative infrastructure. Because synchronization occurs through ordered command replay rather than full state diffing, the volume of data transmitted on reconnection is proportional to the number of offline actions rather than the total world size.

The approach introduces its own requirements: commands must be deterministic and replay-safe, ordering must be preserved across the command log, and server-side validation logic must cover all command types to prevent exploita-

tion. These constraints are manageable within the scope of the existing architecture but would require careful engineering in a production system.

Overall, the results suggest that a command-based synchronization model provides a practical and extensible foundation for hybrid offline–online persistence in survival video games, and that consistent synchronization can be achieved without fundamentally redesigning the existing server-authoritative framework.

References

Baker, N. (2025). *Online gaming statistics*. https://www.uswitch.com/broadband/studies/online-gaming-statistics/

Bernstein, P. A., & Goodman, N. (1981). Concurrency control in distributed database systems. *ACM Computing Surveys*, *13*(2), 185–221. https://doi.org/10.1145/356842.356846

Nystrom, R. (2014). *Game programming patterns*. Genever — Benning. https://books.google.no/books?id=9fIwBQAAQBAJ

Ondruška, D., Scott, R., Katzner, M.-M., Cerezo, A. H., & Barczyk, M. (2026). *Stop killing games at the european parliament*. https://www.youtube.com/watch?v=QXdmoeaYZ9Y